



Betingelser og logik

Program

- Betingelser og logik
- Øvelser
- Opsamling

Enten/eller - if og else



WORK WORK WORK WORK

En normal (kedelig) hverdag

```
public class MyRoutine {  
    public static void main(String[] args) {  
        wakeUp();  
        work();  
        goToBed();  
    }  
}
```

```
wakeUp();  
work();  
goToBed();
```




Demo - weekend



Flere betingelser - else if



I gamle dage (før aircondition) kunne man få varmefri...

```
boolean isWeekend = true; // Er det weekend?  
int temp = 31; // Temperaturen udenfor  
  
wakeUp();  
if (temp > 40) {  
    goToBeach();  
} else if (isWeekend) {  
    watchNetflix();  
} else {  
    commute();  
    work();  
    commute();  
}
```

Indlejrrede betingelser

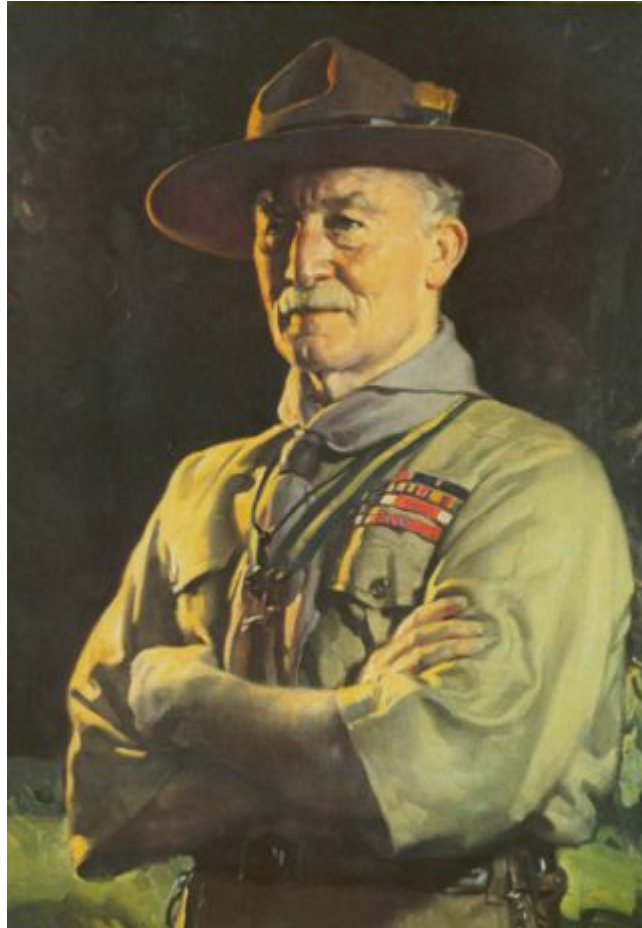
I dag kan vi ikke få varmfri, så...

```
boolean isWeekend = true; // Er det weekend?  
int temp = 31; // Temperaturen udenfor  
  
wakeUp();  
if (isWeekend) {  
    if (temp > 30) {  
        goToBeach();  
    } else {  
        watchNetflix();  
    }  
} else {  
    commute();  
    work();  
    commute();  
}  
goToBed();
```

... med mindre det regner - eller jeg er syg ... argh!

```
boolean isWeekend = true; // Er det weekend?
int temp = 31; // Temperaturen udenfor
boolean isRaining = true; // Regner det?
boolean isSick = false; // Er jeg syg?

wakeUp();
if (isWeekend) {
    if (temp > 30) {
        if (isRaining) {
            watchNetflix();
        } else {
            if (isSick) {
                watchNetflix();
            } else {
                goToBeach();
            }
        }
    } else {
        watchNetflix();
    }
} else {
    commute();
    work();
    commute();
}
goToBed();
```



Efterlad denne verden kode bedre end du fandt den
- Baden-Powell

Eller...

Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live. Code for readability.

- Martin Golding

Logiske operatorer

Kan I huske, at vi talte om **operatorer** i Java?

- Aritmetiske operatorer: +, -, *, /
 - `fx. 5 - 3;`
- Sammenligningsoperatorer: ==, !=, <, >, <=, >=
 - `fx.temp >= 100;`
- Tildelingsoperatorer: =, +=, -=, *=, /=
 - `fx.x += 3;`
- Logiske operatorer: &&, ||, !
 - `fx.raining || bringingUmbrella;`

Logiske operatører arbejder med værdier, der er enten **true** eller **false**

Vi kan kombinere betingelser med logiske operatorer

Se Netflix er:

- Det er *weekend*
- **og**
 - *Det regner*
eller
 - *Jeg er syg*
eller
 - *Temperaturen er max 30 grader*

```
isWeekend && (isRaining || isSick || temp <= 30)
```

```
...  
if (isWeekend && (isRaining || isSick || temp <= 30)) {  
    watchNetflix();  
else if (isWeekend && temp > 30) {  
    goToBeach();  
} else {  
    commute();  
    work();  
    commute();  
}  
...
```


Nogle gange skal man bare vende logikken om

```
...  
if (!isWeekend) {  
    commute();  
    work();  
    commute();  
} else if (isSick || isRaining || temp < 30) {  
    watchNetflix();  
} else {  
    goToBeach();  
}  
...
```

Om et års tid har du glemt, hvorfor

```
isSick || isRaining || temp < 30
```

betyder "se Netflix"

Navngivning gør det nemmere at forstå

```
...  
boolean isBeachWeather = temp > 30 && !isRaining;  
  
if (!isWeekend) {  
    commute();  
    work();  
    commute();  
}  
else if (isBeachWeather) {  
    goToBeach();  
} else {  
    watchNetflix();  
}  
...
```

Vi kan også lave en metode, der tjekker om det er strandvejr

```
if (!isWeekend) {  
    commute();  
    work();  
    commute();  
}  
else if (isBeachWeather(temp, isRaining)) {  
    goToBeach();  
} else {  
    watchNetflix();  
}  
  
public static boolean isBeachWeather(int temp, boolean isRaining) {  
    return temp > 30 && !isRaining;  
}
```

Vi *kunne* også bruge **if** og **else** i metoden

```
public static boolean isBeachWeather(int temp, boolean isRaining) {  
    boolean isBeachWeather;  
  
    if (temp > 30) {  
        if (!isRaining) {  
            isBeachWeather = true;  
        } else {  
            isBeachWeather = false;  
        }  
    } else {  
        isBeachWeather = false;  
    }  
    return isBeachWeather;  
}
```

men det er i virkeligheden nemmere "springe ud af funktionen"

```
public static boolean isBeachWeather(int temp, boolean isRaining) {  
    if (temp > 30) {  
        if (!isRaining) {  
            return true; // varmt og ikke regner  
        } else {  
            return false; // varmt og regner  
        }  
    } else {  
        return false; // koldt  
    }  
}
```

En endnu mere læsbare måde er at anvende **guards**,

her undgår vi helt indlejrede if/else

Guards


```
public isBeachWeather(int temp, boolean isRaining) {  
    if (temp <= 30) {  
        return false; // koldt  
    }  
  
    if (isRaining) {  
        return false; // det regner  
    }  
  
    return true; // Ellers er det strandvejr  
}
```

Switch

```
public static boolean isWeekend(dayOfWeek) {  
    boolean isWeekend;  
  
    switch (dayOfWeek) {  
        case "Monday":  
            isWeekend = false;  
            break;  
        case "Tuesday":  
            isWeekend = false;  
            break;  
        case "Wednesday":  
            isWeekend = false;  
            break;  
        case "Thursday":  
            isWeekend = false;  
            break;  
        case "Friday":  
            isWeekend = false;  
            break;  
        case "Saturday":  
            isWeekend = true;  
            break;  
        case "Sunday":  
            isWeekend = true;  
            break;  
    }  
    return isWeekend;  
}
```

flere case'er kan samles

```
public static boolean isWeekend(dayOfWeek) {  
    boolean isWeekend;  
  
    switch (dayOfWeek) {  
        case "Monday":  
        case "Tuesday":  
        case "Wednesday":  
        case "Thursday":  
        case "Friday":  
            isWeekend = false;  
            break;  
        case "Saturday":  
        case "Sunday":  
            isWeekend = true;  
            break;  
    }  
    return isWeekend;  
}
```

default case

```
public static boolean isWeekend(dayOfWeek) {
    boolean isWeekend;

    switch (dayOfWeek) {
        case "Monday":
        case "Tuesday":
        case "Wednesday":
        case "Thursday":
        case "Friday":
            isWeekend = false;
            break;
        case "Saturday":
        case "Sunday":
            isWeekend = true;
            break;
        default:
            System.out.println("Ugyldig ugedag: " + dayOfWeek);
            isWeekend = false; // eller throw exception
            break;
    }
    return isWeekend;
}
```

Ligesom i **if**-sætninger kan vi i en metode returnere en værdi direkte

```
public static boolean isWeekend(String dayOfWeek) {  
    switch (dayOfWeek) {  
        case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday":  
            return false;  
        case "Saturday", "Sunday":  
            return true;  
        default:  
            return false;  
    }  
}
```

I nyere Java, **switch** expressions

```
public static boolean isWeekend(String dayOfWeek) {  
    return switch (dayOfWeek) {  
        case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" -> false;  
        case "Saturday", "Sunday" -> true;  
        default -> false;  
    };  
}
```

Ternary operator


```
boolean beachWeather = temp > 30 && !isRaining ? true : false;
```

dvs.

betingelse ? **værdi hvis sand** : **værdi hvis falsk**

```
boolean goToBeach = isWeekend ? isBeachWeather(temp, isRaining) : false;
```

- Det er en kompakt måde at skrive betingelser på, men det kan gøre koden mindre læsbar, hvis den bliver for kompleks.



Tre ting du tager med fra fra i dag?